

OriginAnecdote

Developer's Log & System Documentation

Project Title:	OriginAnecdote International Scheduling Web Application
Developer:	Antigravity (AI Coding Assistant by Google DeepMind)
Client:	ESL / Language Instructors & Teachers
Date:	June 25, 2026
Status:	Completed & Hosted Live

1. Executive Summary

OriginAnecdote is a premium, client-side single-page dashboard application built specifically for educators conducting classes for international students. The project tackles timezone differences and calendar visibility issues associated with global tutoring, offering booking request verification, automated local student clock checks, dynamic timezone simulation, and visual student analytics.

2. Research & Requirements Phase

Before starting development, research was conducted on leading online tutor dashboards such as Preply and italki. These platforms provide teachers with:

- **Automatic Timezone Shifting:** Translating slots automatically based on student and teacher account regions.
- **Recurring Weekly Availabilities:** Allowing teachers to open set hours to structure their weeks.
- **Safety Notice Policies:** Preventing last-minute bookings (e.g. requiring 12-24 hours notice) to avoid scheduling conflicts.

OriginAnecdote addresses these areas with specific frontend modules:

1. An **Interactive Timezone Simulator** allowing the teacher to view their calendar from the perspective of students in Tokyo, São Paulo, Paris, London, New York, or Beijing.
2. An **Alert Helper** indicating whether a proposed student request lands in a reasonable study hour (daytime) or early morning/late night for the student.
3. **Live Student Clocks** displaying real-time local times on CRM cards, complete with visual alerts indicating when students are likely sleeping.

3. System Architecture & Tech Stack

To ensure instantaneous load times and simple deployment, the system is designed as a standalone static web application:

- **HTML5 structure:** Semantic markup dividing the viewport into a sticky navigation sidebar and dynamic content views.
- **Vanilla CSS (Responsive Layouts):** A global design system using HSL color variables. Uses a dark glassmorphism styling approach, containing linear gradients, backdrop-filters, custom scrollbars, and micro-animations.
- **Vanilla JavaScript (ES6 Modules):** Client-side model-view-controller (MVC) logic. Manages states for calendar availability, timezone calculation offsets, notifications, student search queries, and modal rescheduling dialogs.

4. Core Code Modules

4.1 HTML Structure (index.html)

Defines the sidebar, headers, and five distinct view panels:

- **Dashboard:** Displays metric summaries, the upcoming active classroom card, and the booking requests list.
- **Schedule & Slots:** Embeds the week grid and availability controls.
- **My Students:** Lists student details with search and level filters.
- **Earnings & Insights:** Features monthly earnings history and timezone demand progress bars.
- **Settings:** Contains account fields and video classroom integrations.

4.2 CSS Design Language (style.css)

Implements a responsive, dark glassmorphism system using high-end HSL styling variables:

```
:root {  
  --bg-base: hsl(222, 47%, 8%); /* Deep space blue */  
  --panel-bg: hsla(222, 30%, 15%, 0.55);  
  --color-primary: hsl(174, 90%, 42%); /* Neon Teal */  
  --color-secondary: hsl(262, 85%, 66%); /* Neon Purple */  
  --color-success: hsl(145, 80%, 45%); /* Neon Green */  
  --color-warning: hsl(38, 95%, 55%); /* Gold */  
  --color-danger: hsl(355, 85%, 55%); /* Coral Rose */  
}
```

4.3 JavaScript State Controller (app.js)

Coordinates time translations, updates clocks every second, and updates availability grids dynamically. Here is the timezone translation helper:

```
function translateTime(hour, sourceTz, targetTz) {  
  const sourceOffset = TZ_OFFSETS[sourceTz] || 0;  
  const targetOffset = TZ_OFFSETS[targetTz] || 0;  
  const diff = targetOffset - sourceOffset;  
  
  let translatedHour = hour + diff;  
  let dayOffset = 0;  
  
  if (translatedHour >= 24) {  
    translatedHour -= 24;  
    dayOffset = 1;  
  } else if (translatedHour < 0) {  
    translatedHour += 24;  
    dayOffset = -1;  
  }  
  return { hour: translatedHour, dayOffset: dayOffset };  
}
```

5. Step-by-Step Build Chronology

1. **Research & Planning:** Analyzed teacher scheduling requirements and submitted an implementation plan.
2. **Structural Skeleton:** Built the standard single-page app structure (`index.html`) using semantic divisions.
3. **Visual System Polish:** Injected variables and responsive grids in `style.css` using backdrop-filter blur definitions.
4. **State Controls:** Linked standard JS state logic (`app.js`) to generate calendar slots dynamically and enable active timezone transformations.
5. **Local Verification:** Deployed a browser subagent locally to test tab controls, slot toggles, booking approvals, and notes storage.
6. **GitHub Pages Hosting:** Configured local Git, created the GitHub repository `Augmencord/OriginAnecdote`, pushed the main branch, and triggered the pages deployment via API.
7. **Live Verification:** Re-ran the browser test directly against the public URL to verify assets.

6. Compiled Prompts Chronicle

Below are all the prompts submitted by the user to build the application, arranged in chronological order:

Prompt #1 (Initial System Request):

"create a basic front end application of an Edtech in which you have the functionality of booking classes and it shouws your available slots. Its hould be centered around teachers, means the application if for teachers. research about all the available application for teachers specially the one that deals with international students and then make the front end."

Prompt #2 (Application Renaming):

"change the name of the application to OriginAnecdote rest follow the same implementation plan as mentioned above"

Prompt #3 (Hosting Request):

"host it on github if github provides free hosting for a web application that you created"

Prompt #4 (Documentation & PDF Compilation):

"create a developers log Pdf documentation in the same folder mentioning the process of how you built this frontend application. Also compile all the prompts I sent."